

Aula 3 — Gramáticas Livres de Contexto e Gramáticas Ambíguas

Compiladores I

FACOM - Faculdade de Computação

Bloco 2: Fundamentos

Duração estimada: 70–80 minutos

Revisão da Aula 2

- ▶ Fases do compilador, entradas e saídas;
- ▶ Sintaxe define forma; semântica define significado.

Pergunta rápida

Qual fase do compilador é responsável por verificar se a *estrutura* de um programa está correta?

Objetivos de aprendizagem

- ▶ Definir formalmente uma gramática livre de contexto (GLC);
- ▶ Construir derivações e árvores de derivação para sentenças simples;
- ▶ Escrever gramáticas em BNF/EBNF;
- ▶ Identificar e corrigir gramáticas ambíguas.

Sumário da aula

Gramáticas livres de contexto

Gramáticas ambíguas

Definição formal de GLC

Uma gramática $G = (N, T, P, S)$

- ▶ N : conjunto de não-terminais (símbolos que se expandem);
- ▶ T : conjunto de terminais (símbolos finais, ex: tokens);
- ▶ P : conjunto de produções da forma $A \rightarrow \alpha$, $A \in N$;
- ▶ $S \in N$: símbolo inicial.

Exemplo: gramática de expressões do Micro C

Exemplo — Notação BNF

```
1  expr    -> expr '+' termo | expr '-' termo | termo
2  termo   -> termo '*' fator | termo '/' fator | fator
3  fator   -> NUM | ID | '(' expr ')'
```

Essa é (uma versão simplificada de) parte da gramática usada no Capítulo 3 do Micro C para expressões aritméticas.

Derivações e árvores de derivação

- ▶ Uma **derivação** é uma sequência de substituições de não-terminais por produções, começando em S ;
- ▶ A **árvore de derivação** (ou árvore sintática) representa visualmente essas substituições;
- ▶ A linguagem $L(G)$ é o conjunto de todas as sentenças de terminais deriváveis a partir de S .

Pergunta rápida

Usando a gramática de *expr* acima, derive a sentença $ID + NUM * ID$. Quantos passos você precisou?

Solução: derivação de $ID + NUM * ID$

Solução padrão

Derivação mais à esquerda (leftmost) — 8 passos

1	expr	(1) =>	expr '+' termo	(5) =>	ID '+' termo '*' fator
2		(2) =>	termo '+' termo	(6) =>	ID '+' fator '*' fator
3		(3) =>	fator '+' termo	(7) =>	ID '+' NUM '*' fator
4		(4) =>	ID '+' termo	(8) =>	ID '+' NUM '*' ID

Para discutir

- ▶ Por que termo nunca se expande direto para expr? O que isso garante sobre a precedência de $*$ em relação a $+$?
- ▶ Existe uma derivação *rightmost* diferente? Ela produz a mesma árvore?

Notação BNF/EBNF

- ▶ **BNF**: cada produção é $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots$;
- ▶ **EBNF**: acrescenta operadores de repetição/opcionalidade: `[]` (opcional), `{ }` (zero ou mais), `( )` (agrupamento);
- ▶ Muitas linguagens reais (como a gramática completa do Micro C, no Apêndice B do livro) são especificadas em EBNF por concisão.

Exemplo —

`bloco -> '{' { comando } '}'` descreve um bloco com zero ou mais comandos.

Exercício: convertendo EBNF para BNF

Exercício

A produção EBNF abaixo descreve uma lista de parâmetros:

1 `lista_param -> ID { ', ' ID }`

Reescreva essa produção em BNF pura (sem { }), introduzindo um novo não-terminal se necessário.

Solução: convertendo EBNF para BNF

Solução padrão

```
1 lista_param -> ID resto_param
2 resto_param -> ', ' ID resto_param
3               | (* vazio *)
```

Para discutir

- ▶ Essa não é a única solução correta! Quem escreveu com recursão à esquerda (`resto_param -> resto_param ', ' ID`) também está certo — qual é a diferença prática entre as duas versões?
- ▶ O que aconteceria se esquecêssemos a alternativa vazia em `resto_param`?

Sumário da aula

Gramáticas livres de contexto

Gramáticas ambíguas

O problema da ambiguidade

Definição

Uma gramática é **ambígua** se existe pelo menos uma sentença com duas ou mais árvores de derivação diferentes.

Exemplo — Gramática ambígua clássica

$\text{expr} \rightarrow \text{expr} \text{'+'} \text{expr} \mid \text{expr} \text{'*'} \text{expr} \mid \text{ID}$

A sentença $\text{ID} + \text{ID} * \text{ID}$ pode ser derivada de duas formas diferentes!

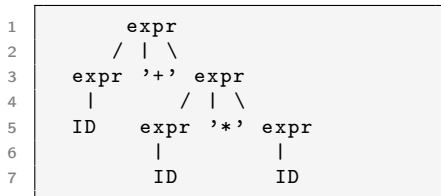
Pergunta rápida

Desenhe (no papel/quadro) as duas árvores de derivação possíveis para $\text{ID} + \text{ID} * \text{ID}$ usando essa gramática.

Solução: as duas árvores de $ID + ID * ID$

Solução padrão

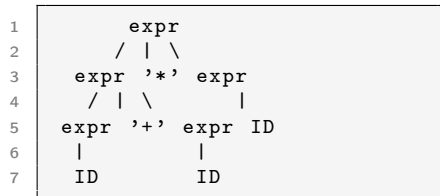
Árvore A — raiz '+'



Leitura: $ID + (ID * ID)$

Solução padrão

Árvore B — raiz '*'



Leitura: $(ID + ID) * ID$

Para discutir

- ▶ Qual árvore corresponde à precedência usual da aritmética (multiplicação antes da adição)?
- ▶ Como a gramática *expr/termo/fator* vista antes evita esse problema *estruturalmente*, sem regras extras de precedência?

Corrigindo a ambiguidade

- ▶ Reescrevemos a gramática introduzindo **camadas de precedência** (como fizemos com `expr/termo/fator`);
- ▶ Cada camada representa um nível de precedência: operadores de menor precedência ficam "por fora", os de maior precedência "por dentro";
- ▶ A **associatividade** (esquerda/direita) é resolvida pela posição da recursão na produção (recursão à esquerda $\Rightarrow$ associa à esquerda).

Desafio em sala

Desafio em sala

A gramática abaixo é ambígua. Mostre que $5 - 3 - 1$ pode gerar dois resultados diferentes, dependendo da árvore, e reescreva a gramática para que a subtração seja sempre associativa à esquerda (resultado correto: 1).

$$\text{expr} \rightarrow \text{expr} \text{ '-' } \text{expr} \mid \text{NUM}$$

Solução: as duas leituras de $5 - 3 - 1$

Solução padrão

Associando à esquerda

1	$(5 - 3) - 1$
2	$= 2 - 1$
3	$= 1$

Solução padrão

Associando à direita

1	$5 - (3 - 1)$
2	$= 5 - 2$
3	$= 3$

Para discutir

- Qual das duas árvores acima é a "correta" segundo a convenção matemática usual de subtração?

Solução: gramática corrigida para 5 - 3 - 1

Solução padrão

Gramática corrigida (associativa à esquerda)

1 `expr -> expr '-' NUM | NUM`

A recursão à esquerda força cada novo '-' a se apoiar no resultado acumulado anterior, reproduzindo a leitura $((5 - 3) - 1) = 1$.

Para discutir

- ▶ Por que a subtração *precisa* ser associativa à esquerda, ao contrário de operadores como + ou *, cuja associatividade não afeta o resultado final?
- ▶ O que aconteceria se, por engano, escrevêssemos `expr -> NUM '-' expr`? Essa gramática ainda seria ambígua?

Exercício: ambiguidade do *dangling else*

Exercício

Considere a gramática simplificada de comandos condicionais:

```
1 cmd -> 'if' expr 'then' cmd
2       | 'if' expr 'then' cmd 'else' cmd
3       | outro
```

Mostre que a sentença

if E1 then if E2 then S1 else S2

é ambígua: a que if pertence o else, ao primeiro ou ao segundo?

Solução: as duas leituras do *dangling else*

Solução padrão

Leitura A else pertence ao if E2 mais interno:

```
if E1 then (if E2 then S1 else S2)
```

Solução padrão

Leitura B else pertence ao if E1 mais externo:

```
if E1 then (if E2 then S1) else S2
```

Para discutir

- ▶ A gramática original, tal como escrita, não indica qual das duas leituras é a pretendida — por isso é ambígua.
- ▶ Qual leitura parece mais "natural" para quem programa? Por quê?

Solução: gramática corrigida para o *dangling else*

Solução padrão

Convenção padrão (adotada por C, Java, Micro C, ...) O else sempre casa com o if **aberto** mais próximo (Leitura A). A gramática é reescrita distinguindo comandos *fechados* (sem if pendente) de *abertos*:

```
1 cmd          -> cmd_fechado | cmd_aberto
2 cmd_fechado -> 'if' expr 'then' cmd_fechado
3              'else' cmd_fechado
4              | outro
5 cmd_aberto  -> 'if' expr 'then' cmd
6              | 'if' expr 'then' cmd_fechado
7              'else' cmd_aberto
```

Para discutir

- ▶ Por que essa reescrita elimina a ambiguidade sem mudar a linguagem reconhecida?
- ▶ Você conhece alguma linguagem que resolve esse problema de outro jeito (ex.: exigindo chaves ou fim if)? Quais as vantagens e desvantagens?

Síntese da aula

- ▶ Uma GLC é definida por (N, T, P, S) ;
- ▶ Derivações e árvores de derivação mostram como uma sentença é gerada;
- ▶ Gramáticas ambíguas têm mais de uma árvore para a mesma sentença e devem ser reescritas usando níveis de precedência e recursão orientada à associatividade.

Próxima aula: definições regulares e esquemas de tradução.

Teste rápido — Aula 3

Questões teóricas

1. O que torna uma gramática ambígua? Dê um exemplo.
2. Como a recursão à esquerda em uma produção influencia a associatividade do operador?

Questão prática

3. Escreva uma GLC (em BNF) não ambígua para expressões que usam apenas os operadores + e *, respeitando a precedência usual da aritmética.